

Ciência da Computação · Turma CC6NA
Linguagens Formais e Autômatos

ValidaFrete

Validador de Cadastros de Fretes com Expressões Regulares

Relatório Técnico — Trabalho do 1º Bimestre

Yuri Aguiar

Pedro Paulo

João Rath

30 de setembro de 2026

<https://github.com/yuriaguiardev/valida-frete>

Sumário

1. Introdução
2. Problema e objetivos
3. Fundamentação teórica
4. Desenvolvimento da aplicação
5. Expressões Regulares implementadas (ER-01 a ER-05)
6. Testes e análise dos resultados
7. Limitações e melhorias futuras
8. Contribuições dos integrantes
9. Uso de Inteligência Artificial
10. Conclusão
11. Referências

Apêndice A — Tabelas de transição dos AFNe

1. Introdução

Expressões Regulares (ERs) são uma notação algébrica para descrever linguagens regulares e estão presentes em praticamente todas as linguagens de programação como ferramenta de validação, busca e extração de texto. Este trabalho relaciona a teoria estudada em Linguagens Formais e Autômatos — ERs, linguagens regulares e autômatos finitos não determinísticos com movimentos vazios (AFNe) — com a construção de uma aplicação funcional.

O **ValidaFrete** é uma aplicação web escrita em Python que valida os campos de cadastros de fretes usando cinco Expressões Regulares. Um diferencial do projeto é que o AFNe de cada expressão não foi desenhado à mão: ele é **gerado automaticamente a partir do padrão escrito no código**, pela construção de Thompson, e os testes automatizados comprovam que o autômato e o motor `re` do Python aceitam e rejeitam exatamente as mesmas cadeias. Assim, a ER formal, a sintaxe do código, os testes e o AFNe representam a mesma linguagem, como exige a regra central do trabalho.

2. Problema e objetivos

Transportadoras de pequeno e médio porte recebem cadastros de fretes preenchidos manualmente por filiais, parceiros e clientes, em planilhas. É comum que esses dados cheguem com erros de formatação: CNPJ com máscara incompleta, placas em letras minúsculas ou com hífen no padrão Mercosul, telefones com DDD inexistente, datas impossíveis (como 31/04) e valores monetários escritos de formas diferentes. Esses erros só são percebidos quando a emissão do documento fiscal é recusada ou quando o motorista não é localizado, gerando retrabalho e atrasos.

Em 2026 o problema se agravou com a entrada em vigor do **CNPJ alfanumérico** (Instrução Normativa RFB nº 2.229/2024): sistemas que aceitavam apenas dígitos passaram a rejeitar

cadastros válidos, enquanto validações improvisadas aceitam valores inválidos.

Objetivo geral

Desenvolver uma aplicação que valide automaticamente os campos de cadastros de fretes por meio de Expressões Regulares, indicando de forma clara quais valores estão incorretos e por quê.

Objetivos específicos

- Definir cinco linguagens regulares relevantes ao domínio e descrevê-las por ERs na notação formal.
- Implementar as ERs em Python com correspondência completa (`re.fullmatch`).
- Construir o AFNe correspondente a cada ER e usá-lo para diagnosticar a posição da falha.
- Processar planilhas CSV inteiras, produzindo resumo e relatório de inconsistências.
- Tratar entradas vazias ou inválidas com mensagens claras ao usuário.
- Testar cada ER com no mínimo seis cadeias aceitas, seis rejeitadas e casos-limite.

3. Fundamentação teórica

3.1 Alfabetos, cadeias e linguagens

Um **alfabeto** Σ é um conjunto finito e não vazio de símbolos. Uma **cadeia** sobre Σ é uma sequência finita de símbolos de Σ ; a cadeia de comprimento zero é a palavra vazia ϵ . Uma **linguagem** é qualquer conjunto de cadeias sobre Σ . Neste trabalho, cada ER possui um alfabeto próprio, definido explicitamente na sua ficha; qualquer símbolo fora de Σ (letras minúsculas, espaços indevidos etc.) leva à rejeição.

3.2 Expressões Regulares

As ERs são definidas indutivamente: $\emptyset$, ϵ e cada símbolo $a \in \Sigma$ são ERs; se r e s são ERs, então $r \mid s$ (união), rs (concatenação) e r^* (fecho de Kleene) também são. Os operadores derivados usados nas implementações são abreviações desses três:

Operador	Definição por operadores básicos
r^+	$r r^*$ (uma ou mais repetições)
$r?$	$(r \mid \epsilon)$ (zero ou uma ocorrência)
$r\{m\}$	$r r \dots r$ (m cópias concatenadas)
$r\{m, n\}$	$r^m \mid r^{m+1} \mid \dots \mid r^n$
$[abc], [a-c]$	$(a \mid b \mid c)$ — classe ou intervalo finito

A precedência adotada é: repetição ($^+ ? \{ \}$) antes da concatenação, e concatenação antes da união; parênteses alteram essa ordem.

3.3 AFN ϵ e o Teorema de Kleene

Um AFN ϵ é uma 5-upla $M = (Q, \Sigma, \delta, q_0, F)$, em que Q é o conjunto finito de estados, $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$ é a função de transição, q_0 é o estado inicial e $F \subseteq Q$ o conjunto de estados finais. O **ϵ -fecho** de um estado é o conjunto de estados alcançáveis apenas por movimentos vazios. Uma cadeia w é aceita quando, partindo do ϵ -fecho de q_0 e processando os símbolos de w (sempre aplicando o ϵ -fecho após cada símbolo), o conjunto final de estados contém algum estado de F .

O **Teorema de Kleene** garante que as linguagens descritas por ERs são exatamente as aceitas por autômatos finitos. Por isso, restringimos as ERs aos operadores regulares: retroreferências ($\backslash 1$), lookahead e recursão foram proibidos, pois podem descrever linguagens não regulares e impedem a correspondência direta com um AFN ϵ .

3.4 Construção de Thompson

A construção de Thompson transforma uma ER em um AFN ϵ de forma recursiva, montando um fragmento para cada subexpressão, com um único estado inicial (sem arestas de entrada) e um único estado final (sem arestas de saída):

- **Símbolo a :** dois estados ligados por uma transição a .

- **União $r \mid s$:** novo estado inicial com movimentos ϵ para os fragmentos de r e s , e movimentos ϵ dos finais deles para um novo estado final.
- **Concatenação rs :** o final de r é ligado ao inicial de s .
- **Fecho r^* :** movimentos ϵ permitem pular r , repeti-lo (do final de volta ao início) ou sair.

Na implementação, a concatenação funde o final de r com o inicial de s (em vez de usar um movimento ϵ), e r^* é representado por um único movimento ϵ que "pula" o fragmento de r . Ambas as simplificações preservam a linguagem, porque o estado inicial de um fragmento de Thompson não recebe arestas e o final não emite arestas, e reduzem o tamanho dos diagramas.

4. Desenvolvimento da aplicação

4.1 Tecnologias

Tecnologia	Uso
Python 3.10+	Linguagem de programação
Módulo <code>re</code> (biblioteca padrão)	Motor de ERs; todas as validações usam <code>re.fullmatch</code>
Flask 3	Interface web local
pytest	Testes automatizados
Graphviz	Desenho dos AFNε a partir dos arquivos DOT gerados
Git / GitHub, VS Code	Versionamento e ambiente de desenvolvimento

4.2 Organização do código

Módulo	Responsabilidade
<code>expressoes.py</code>	Fonte única das cinco ERs: nome, alfabeto, linguagem, notação formal, padrão do código, equivalências e limitações.
<code>casos_teste.py</code>	16 casos de teste por ER (8 aceitas, 8 rejeitadas, com casos-limite marcados).
<code>afn.py</code>	Analizador do padrão (subconjunto regular), construção de Thompson, ε-fecho, simulação passo a passo e exportação para DOT.
<code>validador.py</code>	Valida um valor com <code>re.fullmatch</code> e, se rejeitado, usa o AFNε para explicar em que posição a cadeia falhou.
<code>processador.py</code>	Lê planilhas CSV (UTF-8 ou Windows-1252, separador <code>;</code> ou <code>,</code>), valida todos os campos e gera o relatório.
<code>app.py</code>	Rotas web: validar valor, processar planilha, fichas das ERs com simulação, testes ao vivo.

Como todas as partes do sistema — validação, página web, testes, diagramas e esta documentação — leem as ERs do mesmo módulo, não há cópias que possam divergir.

4.3 Entradas, processamento e saídas

Entrada	Processamento	Saída
Valor digitado e tipo do campo (ou "identificar automaticamente")	<code>re.fullmatch</code> com a ER do campo (ou com as cinco); diagnóstico pelo AFNε	Aceita/rejeitada e mensagem com a posição e o símbolo que causaram a rejeição
Planilha CSV com as colunas <code>id;cnj;placa;telefone;data;valor</code>	Validação de cada campo de cada registro	Totais, aceitas/rejeitadas por ER, tabela com erros destacados e relatório CSV para download

Cadeia na ficha de uma ER	Simulação do AFNε	Conjunto de estados ativos após cada símbolo e resultado do AFNε e do re
---------------------------	-------------------	--

4.4 Tratamento de entradas vazias ou inválidas

- **Valor vazio:** mensagem "Entrada vazia" — ϵ não pertence a nenhuma das linguagens.
- **Valor longo demais** (mais de 100 caracteres) ou tipo de campo inexistente: mensagem de erro, sem processamento.
- **Símbolo fora de Σ :** a mensagem indica o símbolo e a posição, com dica específica para espaços sobrando.
- **Cadeia incompleta:** quando toda a entrada é lida sem atingir o estado final, o programa informa que faltam símbolos.
- **Arquivo:** nenhum arquivo selecionado, extensão diferente de .csv, arquivo vazio, maior que 1 MB, binário, com codificação ilegível, sem registros, com cabeçalho sem as colunas exigidas ou com colunas repetidas — cada caso tem uma mensagem própria.
- **Linha com número de colunas diferente do cabeçalho:** o registro é marcado como erro estrutural, e o restante do arquivo continua sendo processado.
- **Espaços extras não são removidos** automaticamente: " 11222333000181" é rejeitado, exatamente como nos testes, para que o programa reconheça a mesma linguagem da ER.

5. Expressões Regulares implementadas

As convenções da notação formal usadas nas fichas são: justaposição representa concatenação; | representa união; $r\{m\}$ e $r\{m,n\}$ são repetições; símbolos de pontuação aparecem entre aspas simples ('.', '/', '-', ') para não se confundirem com operadores; $_$ representa o espaço; as letras D, P, M, A, F e K são conjuntos ou abreviações definidos no alfabeto de cada ficha. Em todas as ERs a correspondência é completa (re.fullmatch), equivalente a ancorar o padrão com ^ e \$.

ER	Campo	Estados	Transições	Mov. ε
ER-01	CNPJ do cliente (alfanumérico ou numérico)	36	36	4
ER-02	Placa do veículo (padrão antigo ou Mercosul)	16	17	5
ER-03	Telefone do motorista (fixo ou celular, com DDD válido)	84	105	48
ER-04	Data da coleta (DD/MM/AAAA com dia compatível com o mês)	50	56	26
ER-05	Valor do frete em reais (R\$)	25	33	17

5.1 ER-01 – CNPJ do cliente (alfanumérico ou numérico)

Identificação	ER-01 · campo cnpj
Finalidade	Validar o CNPJ da empresa que contrata o frete. Aceita o novo CNPJ alfanumérico (IN RFB nº 2.229/2024, em uso desde julho de 2026) e o formato numérico tradicional, com a máscara XX.XXX.XXX/XXXX-DD ou sem máscara.
Alfabeto (Σ)	$D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ (dígitos) $M = \{A, B, C, \dots, Z\}$ (26 letras maiúsculas, sem acento) $A = M \cup D$ (símbolos permitidos na raiz e na ordem do CNPJ) $\Sigma = M \cup D \cup \{'.' , '/' , '-'\}$
Linguagem L	Cadeias com 14 caracteres significativos: os 12 primeiros pertencem a A (letras maiúsculas ou dígitos) e os 2 últimos, os dígitos verificadores, pertencem a D. A cadeia é escrita sem máscara (14 símbolos seguidos) ou com a máscara completa AA.AAA.AAA/AAAA-DD; máscaras parciais não pertencem a L.
ER formal	$A\{2\} \cdot ' ' A\{3\} \cdot ' ' A\{3\} \cdot ' ' A\{4\} \cdot ' ' D\{2\} \mid A\{12\} D\{2\}$
Sintaxe implementada	<pre>re.fullmatch(r"[A-Z0-9]{2}\.[A-Z0-9]{3}\.[A-Z0-9]{3}/[A-Z0-9]{4}-[0-9]{2}\mid[A-Z0-9]{12}[0-9]{2}", cadeia)</pre>

Equivalência entre a sintaxe e os operadores formais

Na sintaxe do código	Operador formal
$[A-Z0-9]$	Classe finita: $A = M \cup D = (A \mid B \mid \dots \mid Z \mid 0 \mid 1 \mid \dots \mid 9)$
$[0-9]$	Intervalo finito: $D = (0 \mid 1 \mid \dots \mid 9)$
$\{2\}, \{3\}, \{4\}, \{12\}$	Repetição exata: $r\{m\} = r r \dots r$ (m cópias concatenadas)
$\backslash \cdot$	Ponto literal $\cdot$ (o escape remove o significado de "qualquer caractere")
$/$ e $-$	Símbolos literais $'/'$ e $'-'$ (fora de classes não são operadores)
$ $	União das duas formas: com máscara $ $ sem máscara
<code>re.fullmatch</code>	Correspondência completa: equivale a ancorar o padrão no início (^) e no fim (\$); a cadeia inteira precisa pertencer a L.

Testes

#	Cadeia	Esperado	Obtido (re / AFNe)	Descrição
1	12.ABC.345/01DE-35	aceita	aceita / aceita	Alfanumérico com máscara (exemplo oficial da Receita Federal)
2	12ABC34501DE35	aceita	aceita / aceita	Alfanumérico sem máscara
3	11.222.333/0001-81	aceita	aceita / aceita	Numérico tradicional com máscara
4	11222333000181	aceita	aceita / aceita	Numérico tradicional sem máscara

5	A1.B2C.3D4/E5F6-07	aceita	aceita / aceita	Letras e dígitos intercalados
6	AB.CDE.FGH/IJKL-12	aceita	aceita / aceita	Raiz e ordem só com letras, com máscara
7	ZZZZZZZZZZZZ99	aceita	aceita / aceita	Limite: maior símbolo de A em todas as 12 posições caso-limite
8	00.000.000/0000-00	aceita	aceita / aceita	Limite: formato válido, DV não conferido (falso positivo conhecido) caso-limite
9	ε (vazia)	rejeita	rejeita / rejeita	Limite: palavra vazia ε caso-limite
10	12.abc.345/01de-35	rejeita	rejeita / rejeita	Letras minúsculas não pertencem a Σ
11	12.ABC.345/01DE-3A	rejeita	rejeita / rejeita	Dígito verificador com letra (deve estar em D)
12	12ABC345/01DE-35	rejeita	rejeita / rejeita	Máscara parcial (faltam os pontos)
13	12.ABC.345/01DE35	rejeita	rejeita / rejeita	Máscara sem o hífen
14	12ABC34501DE3	rejeita	rejeita / rejeita	Limite: 13 caracteres (um a menos) caso-limite
15	12ABC34501DE356	rejeita	rejeita / rejeita	15 caracteres (um a mais)
16	11222333000181	rejeita	rejeita / rejeita	Espaço antes do valor

Resultado e limites

Os 16 casos produziram o resultado esperado em `re.fullmatch` e na simulação do AFNε.

- Valida apenas o formato: os dígitos verificadores (cálculo módulo 11) não são conferidos. Assim, 00.000.000/0000-00 e 11.222.333/0001-00 são aceitos (falso positivo).
- Letras minúsculas são rejeitadas. O programa não converte a entrada para maiúsculas, para não alterar a linguagem reconhecida.
- Máscaras parciais (ex.: 12ABC345/01DE-35) são rejeitadas por decisão de projeto.

5.1 ER-01 — AFNε correspondente

$Q = \{q_0, \dots, q_{35}\}$ (36 estados) · $\Sigma = \{ -, ., /, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z \}$ · estado inicial q_0 (seta "início") · $F = \{q_{32}\}$ (círculo duplo) · 36 transições, 4 movimentos ϵ (tracejados). A tabela δ completa está no Apêndice A.



Figura 1 — AFNε de ER-01, gerado pela construção de Thompson a partir de $[A-Z0-9]\{2\} \setminus . [A-Z0-9]\{3\} \setminus . [A-Z0-9]\{3\} / [A-Z0-9]\{4\} - [0-9]\{2\} | [A-Z0-9]\{12\} [0-9]\{2\}$.

Versão ampliada: docs/afn/er-01-cnpj.svg.

5.2 ER-02 — Placa do veículo (padrão antigo ou Mercosul)

Identificação	ER-02 · campo placa
Finalidade	Validar a placa do caminhão que fará a coleta. Aceita o padrão antigo brasileiro (LLL-NNNN, com hífen opcional) e o padrão Mercosul (LLLNLNN).
Alfabeto (Σ)	$D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ (dígitos) $M = \{A, B, C, \dots, Z\}$ (26 letras maiúsculas, sem acento) $\Sigma = M \cup D \cup \{'\}'$
Linguagem L	Três letras maiúsculas seguidas de: (a) quatro dígitos, com ou sem um hífen entre as letras e os dígitos (padrão antigo); ou (b) dígito, letra e dois dígitos, sem hífen (padrão Mercosul).
ER formal	$M\{3\} (('-' \epsilon) D\{4\} D M D\{2\})$
Sintaxe implementada	<code>re.fullmatch(r"[A-Z]{3}(-?[0-9]{4} [0-9][A-Z][0-9]{2})", cadeia)</code>

Equivalência entre a sintaxe e os operadores formais

Na sintaxe do código	Operador formal
<code>[A-Z]</code>	Intervalo finito: $M = (A B \dots Z)$
<code>[0-9]</code>	Intervalo finito: $D = (0 1 \dots 9)$
<code>{3}</code> , <code>{4}</code> , <code>{2}</code>	Repetição exata (concatenação de cópias)
<code>-?</code>	Opcionalidade: $('-' \epsilon)$
<code>(... ...)</code>	Agrupamento da união entre padrão antigo e padrão Mercosul
<code>re.fullmatch</code>	Correspondência completa: equivale a ancorar o padrão no início (^) e no fim (\$); a cadeia inteira precisa pertencer a L.

Testes

#	Cadeia	Esperado	Obtido (re / AFNe)	Descrição
1	ABC-1234	aceita	aceita / aceita	Padrão antigo com hífen
2	ABC1234	aceita	aceita / aceita	Padrão antigo sem hífen
3	BRA2E19	aceita	aceita / aceita	Padrão Mercosul
4	RI02A18	aceita	aceita / aceita	Padrão Mercosul
5	QWE0Z00	aceita	aceita / aceita	Mercosul com zeros
6	MNO-5678	aceita	aceita / aceita	Padrão antigo com hífen

7	AAA-0000	aceita	aceita / aceita	Limite: menores símbolos em todas as posições caso-limite
8	ZZZ9Z99	aceita	aceita / aceita	Limite: maiores símbolos em todas as posições (Mercosul) caso-limite
9	ε (vazia)	rejeita	rejeita / rejeita	Limite: palavra vazia ε caso-limite
10	abc1d23	rejeita	rejeita / rejeita	Letras minúsculas
11	ABC-1D23	rejeita	rejeita / rejeita	Mercosul não admite hífen
12	AB-1234	rejeita	rejeita / rejeita	Apenas duas letras
13	ABC12345	rejeita	rejeita / rejeita	Cinco dígitos
14	ABC 1234	rejeita	rejeita / rejeita	Espaço no lugar do hífen
15	ABC1DE3	rejeita	rejeita / rejeita	Duas letras na parte final
16	ABC-123	rejeita	rejeita / rejeita	Limite: um dígito a menos caso-limite

Resultado e limites

Os 16 casos produziram o resultado esperado em `re.fullmatch` e na simulação do AFNε.

- Não consulta se a placa existe ou está registrada; apenas o formato é verificado.
- O padrão Mercosul com hífen (ABC-1D23) é rejeitado, pois a placa oficial não o possui.
- Letras minúsculas e espaços são rejeitados.

5.2 ER-02 — AFN ϵ correspondente

$Q = \{q_0, \dots, q_{15}\}$ (16 estados) · $\Sigma = \{ -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z \}$ · estado inicial q_0 (seta "início") · $F = \{q_{15}\}$ (círculo duplo) · 17 transições, 5 movimentos ϵ (tracejados). A tabela δ completa está no Apêndice A.

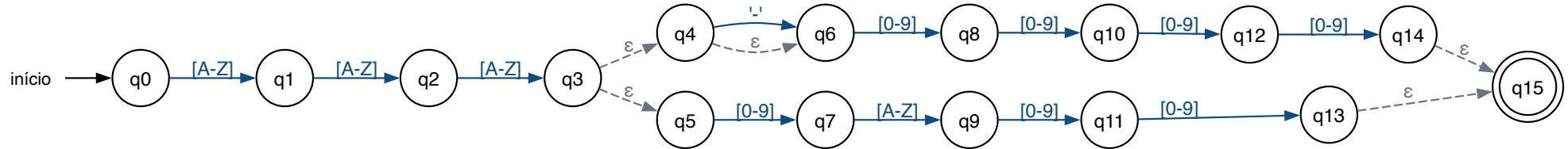


Figura 2 — AFN ϵ de ER-02, gerado pela construção de Thompson a partir de $[A-Z]\{3\}(-?[0-9]\{4\} | [0-9][A-Z][0-9]\{2\})$. Versão ampliada: docs/afn/er-02-placa.svg.

5.3 ER-03 – Telefone do motorista (fixo ou celular, com DDD válido)

Identificação	ER-03 · campo telefone
Finalidade	Validar o telefone de contato do motorista. Exige um DDD realmente em uso no Brasil (67 códigos da Anatel), aceita o prefixo +55 e o DDD com ou sem parênteses — sempre balanceados — e distingue celular (9 + 8 dígitos) de fixo (2 a 5 + 7 dígitos).
Alfabeto (Σ)	$D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ (dígitos) $P = D - \{0\} = \{1, 2, \dots, 9\}$ (dígitos sem o zero) $F = \{2, 3, 4, 5\}$ (primeiro dígito de telefone fixo) $K = 1P \mid 2(1 2 4 7 8) \mid 3(1 2 3 4 5 7 8) \mid 4P \mid 5(1 3 4 5) \mid 6P \mid 7(1 3 4 5 7 9) \mid 8P \mid 9P$ (os 67 DDDs em uso) $\Sigma = D \cup \{ '+', '(', ')', '-', '_ \}$
Linguagem L	Números de telefone formados por: prefixo internacional +55 opcional (seguido ou não de espaço); DDD pertencente a K, entre parênteses ou sem eles; espaço opcional; número de celular (9 seguido de 8 dígitos) ou fixo (dígito de 2 a 5 seguido de 7 dígitos), com hífen opcional antes dos 4 últimos dígitos.
ER formal	$('+' 55 (_ \epsilon) \epsilon) ('(' K ')' K) (_ \epsilon) (9 D\{4\} F D\{3\}) ('-' \epsilon) D\{4\}$
Sintaxe implementada	<code>re.fullmatch(r"(\+55 ?)?(\(((1[1-9] 2[12478] 3[1-578] 4[1-9] 5[1345] 6[1-9] 7[134579] 8[1-9] 9[1-9])\) 1[1-9] 2[12478] 3[1-578] 4[1-9] 5[1345] 6[1-9] 7[134579] 8[1-9] 9[1-9])) ?(9[0-9]{4} [2-5][0-9]{3})-[0-9]{4}", cadeia)</code>

Equivalência entre a sintaxe e os operadores formais

Na sintaxe do código	Operador formal
<code>\+ \ (\)</code>	Símbolos literais '+', '(' e ')' (escapados porque são operadores)
<code>(\+55 ?)?</code>	Opcionalidade aninhada: $('+' 5 5 (_ \epsilon) \epsilon)$
<code>[1-9]</code>	Intervalo: $P = (1 \mid 2 \mid \dots \mid 9)$
<code>[12478], [1-578], [1345], [134579]</code>	Classes finitas: $(1 2 4 7 8), (1 2 3 4 5 7 8), \dots$
<code>1[1-9] 2[12478] ... 9[1-9]</code>	União dos 67 DDDs válidos (abreviada por K na forma formal)
<code>\(K\) K</code>	União que só aceita parênteses balanceados: o K aparece duas vezes, porque uma ER não tem memória para "lembrar" que abriu um parêntese
<code>[2-5]</code>	Intervalo: $F = (2 \mid 3 \mid 4 \mid 5)$
<code>? e -?</code>	Opcionalidade: $(_ \epsilon)$ e $('-' \epsilon)$
<code>re.fullmatch</code>	Correspondência completa: equivale a ancorar o padrão no início (^) e no fim (\$); a cadeia inteira precisa pertencer a L.

Testes

#	Cadeia	Esperado	Obtido (re / AFNε)	Descrição
1	+55_(11)_91234-5678	aceita	aceita / aceita	Celular completo com DDI, DDD entre parênteses e hífen
2	(21)_3456-7890	aceita	aceita / aceita	Fixo com DDD entre parênteses
3	11912345678	aceita	aceita / aceita	Celular apenas com dígitos
4	+5561987654321	aceita	aceita / aceita	DDI e DDD sem separadores
5	(47)98888-0000	aceita	aceita / aceita	Sem espaço após o DDD
6	85_3222-1100	aceita	aceita / aceita	Fixo com DDD sem parênteses
7	+55_31_2345-6789	aceita	aceita / aceita	DDI com DDD sem parênteses
8	(99)_90000-0000	aceita	aceita / aceita	Limite: maior DDD existente caso-limite
9	ε (vazia)	rejeita	rejeita / rejeita	Limite: palavra vazia ε caso-limite
10	(11_91234-5678	rejeita	rejeita / rejeita	Parênteses desbalanceados
11	(23)_91234-5678	rejeita	rejeita / rejeita	DDD 23 não existe
12	11_81234-5678	rejeita	rejeita / rejeita	Nove dígitos sem começar por 9
13	11_6123-4567	rejeita	rejeita / rejeita	Fixo começando por 6
14	+55__11_91234-5678	rejeita	rejeita / rejeita	Dois espaços seguidos
15	11_91234-567	rejeita	rejeita / rejeita	Limite: um dígito a menos caso-limite
16	+1_(11)_91234-5678	rejeita	rejeita / rejeita	DDI diferente de +55

Resultado e limites

Os 16 casos produziram o resultado esperado em `re.fullmatch` e na simulação do AFNε.

- Não verifica se o número está ativo nem se o prefixo pertence a uma operadora.
- Aceita separadores em posições alternativas, como +5511912345678 ou (11)91234-5678, mas rejeita espaços duplos e hífen fora do lugar.
- Números 0800, 0300 e de serviços (190, 192) não pertencem à linguagem.

5.3 ER-03 — AFNε correspondente

$Q = \{q_0, \dots, q_{83}\}$ (84 estados) · $\Sigma = \{ (,), +, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, _ \}$ · estado inicial q_0 (seta "início") · $F = \{q_{83}\}$ (círculo duplo) · 105 transições, 48 movimentos ϵ (tracejados). A tabela δ completa está no Apêndice A.

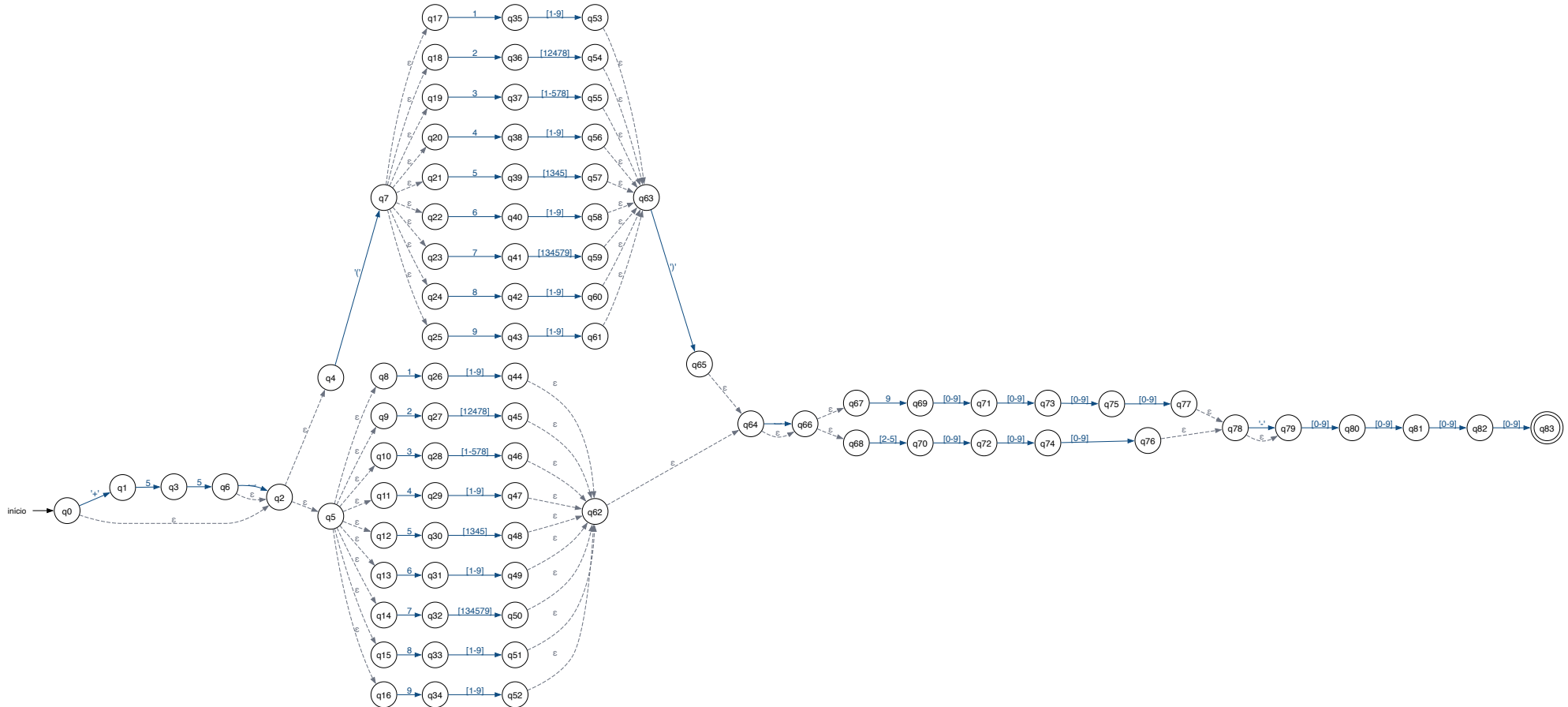


Figura 3 — AFNε de ER-03, gerado pela construção de Thompson a partir de $(\backslash+55\dots$. Versão ampliada: docs/afn/er-03-telefone.svg.

5.4 ER-04 — Data da coleta (DD/MM/AAAA com dia compatível com o mês)

Identificação	ER-04 · campo data
Finalidade	Validar a data agendada para a coleta. Além do formato, impede dias inexistentes como 31/04 ou 30/02, e restringe o ano ao intervalo 1900–2099.
Alfabeto (Σ)	$D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ (dígitos) $P = D - \{0\} = \{1, 2, \dots, 9\}$ (dígitos sem o zero) $\Sigma = D \cup \{'/'\}$
Linguagem L	Datas DD/MM/AAAA com ano entre 1900 e 2099 em que: os dias 01 a 29 ocorrem em qualquer mês; o dia 30 ocorre em todos os meses exceto fevereiro; e o dia 31 ocorre apenas em janeiro, março, maio, julho, agosto, outubro e dezembro.
ER formal	$((0P \mid (1 2)D)' (0P \mid 1(0 1 2)) \mid 30' (0(1 3 4 5 6 7 8 9) \mid 1(0 1 2)) \mid 31' (0(1 3 5 7 8) \mid 1(0 2)))' (19 \mid 20) D\{2\}$
Sintaxe implementada	<pre>re.fullmatch(r"((0[1-9] [12] [0-9]) / (0[1-9] 1[0-2]) 30 / (0[13-9] 1[0-2]) 31 / (0[13578] 1[02])) / (19 20) [0-9]{2}", cadeia)</pre>

Equivalência entre a sintaxe e os operadores formais

Na sintaxe do código	Operador formal
$0[1-9] \mid [12] [0-9]$	Dias 01–29: $(0P \mid (1 2)D)$
$0[1-9] \mid 1[0-2]$	Meses 01–12: $(0P \mid 1(0 1 2))$
$[13-9]$	Classe com intervalo: $(1 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9)$, ou seja, sem o 2 (fevereiro)
$[13578], [02]$	Classes finitas dos meses com 31 dias: $(1 3 5 7 8)$ e $(0 2)$
$19 \mid 20$	União das duas possibilidades de século
$[0-9]\{2\}$	Repetição exata: $D D$
<code>re.fullmatch</code>	Correspondência completa: equivale a ancorar o padrão no início (^) e no fim (\$); a cadeia inteira precisa pertencer a L.

Testes

#	Cadeia	Esperado	Obtido (re / AFNe)	Descrição
1	01/01/2000	aceita	aceita / aceita	Primeiro dia do ano
2	15/08/2025	aceita	aceita / aceita	Data comum
3	29/02/2024	aceita	aceita / aceita	29 de fevereiro em ano bissexto
4	30/04/2026	aceita	aceita / aceita	Dia 30 em mês de 30 dias

5	31/12/1999	aceita	aceita / aceita	Dia 31 em mês de 31 dias
6	31/01/1900	aceita	aceita / aceita	Limite: menor ano aceito caso-limite
7	28/02/2099	aceita	aceita / aceita	Limite: maior ano aceito caso-limite
8	29/02/2023	aceita	aceita / aceita	Limite: aceito pela linguagem, embora 2023 não seja bissexto caso-limite
9	ε (vazia)	rejeita	rejeita / rejeita	Limite: palavra vazia ε caso-limite
10	31/04/2025	rejeita	rejeita / rejeita	Abril não tem dia 31
11	30/02/2024	rejeita	rejeita / rejeita	Fevereiro não tem dia 30
12	00/10/2020	rejeita	rejeita / rejeita	Dia zero
13	15/13/2020	rejeita	rejeita / rejeita	Mês 13
14	1/1/2020	rejeita	rejeita / rejeita	Dia e mês sem zero à esquerda
15	15/08/2100	rejeita	rejeita / rejeita	Limite: ano acima de 2099 caso-limite
16	15-08-2020	rejeita	rejeita / rejeita	Separador '-' em vez de '/'

Resultado e limites

Os 16 casos produziram o resultado esperado em `re.fullmatch` e na simulação do AFNε.

- 29/02 é aceito em qualquer ano (ex.: 29/02/2023, falso positivo). Como o intervalo de anos é finito, os bissextos até poderiam ser descritos por uma ER, mas ela ficaria enorme e ilegível; essa verificação fica para uma etapa posterior.
- Não verifica se a data é passada ou futura, nem aceita formatos como D/M/AAAA ou AAAA-MM-DD.

5.4 ER-04 — AFNε correspondente

$Q = \{q_0, \dots, q_{49}\}$ (50 estados) · $\Sigma = \{/, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ · estado inicial q_0 (seta "início") · $F = \{q_{49}\}$ (círculo duplo) · 56 transições, 26 movimentos ϵ (tracejados). A tabela δ completa está no Apêndice A.

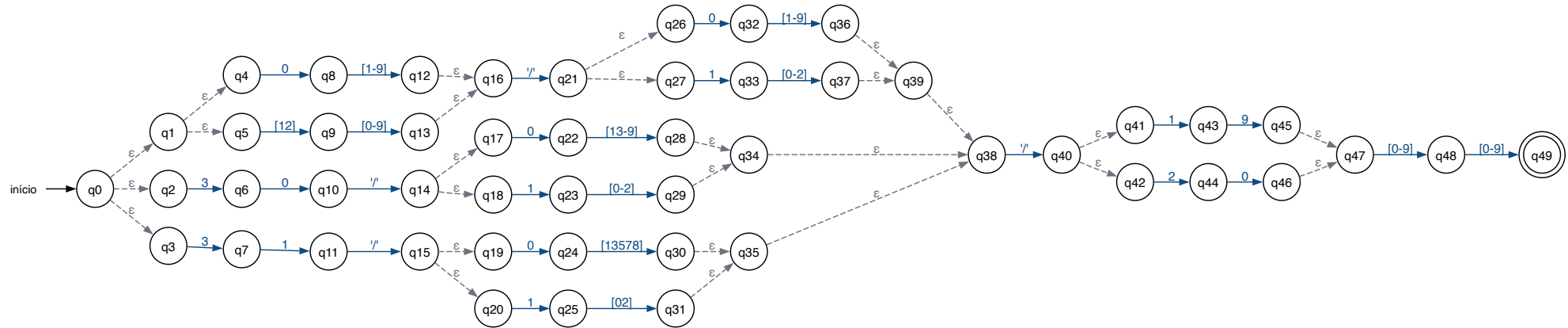


Figura 4 — AFNε de ER-04, gerado pela construção de Thompson a partir de $((\emptyset[1-9] \mid [12][0-9]) / (\emptyset[1-9] \mid 1[0-2])) \mid 30 / (\emptyset[13-9] \mid 1[0-2]) \mid 31 / (\emptyset[13578] \mid 1[02]) / (19 \mid 20)[0-9]\{2\}$. Versão ampliada: docs/afn/er-04-data.svg.

5.5 ER-05 – Valor do frete em reais (R\$)

Identificação	ER-05 · campo valor
Finalidade	Validar o valor cobrado pelo frete no padrão monetário brasileiro: prefixo R\$, parte inteira sem zeros à esquerda, separador de milhar opcional (mas consistente) e exatamente dois centavos após a vírgula.
Alfabeto (Σ)	$D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ (dígitos) $P = D - \{0\} = \{1, 2, \dots, 9\}$ (dígitos sem o zero) $\Sigma = D \cup \{R, '\$', '_', '\.', '\,', '\,'\}$
Linguagem L	Cadeias iniciadas por R\$, com espaço opcional, seguidas da parte inteira — 0, ou um número sem zero à esquerda escrito inteiramente com separadores de milhar '.' em grupos de três dígitos, ou inteiramente sem separadores — e, por fim, vírgula e dois dígitos de centavos.
ER formal	$R '\$' (_ \epsilon) (0 P D\{0,2\} (' D\{3\})^* P D^*) '\,' D\{2\}$
Sintaxe implementada	<code>re.fullmatch(r"R\\$?(0 [1-9][0-9]{0,2}(\.[0-9]{3})* [1-9][0-9]*),[0-9]{2}", cadeia)</code>

Equivalência entre a sintaxe e os operadores formais

Na sintaxe do código	Operador formal
<code>R\\$</code>	Símbolos literais R e '\$' (\$ é escapado porque é âncora de fim no motor)
<code>?</code>	Opcionalidade do espaço: $(_ \epsilon)$
<code>[1-9][0-9]{0,2}</code>	Repetição limitada: $P D\{0,2\} = P (\epsilon D DD)$
<code>(\.[0-9]{3})*</code>	Fecho de Kleene: zero ou mais grupos de milhar '.' D D D
<code>[1-9][0-9]*</code>	Fecho de Kleene: $P D^*$ (inteiro sem separadores)
<code>,[0-9]{2}</code>	Vírgula literal seguida de exatamente dois dígitos
<code>re.fullmatch</code>	Correspondência completa: equivale a ancorar o padrão no início (^) e no fim (\$); a cadeia inteira precisa pertencer a L.

Testes

#	Cadeia	Esperado	Obtido (re / AFNe)	Descrição
1	R\$_1.234,56	aceita	aceita / aceita	Com separador de milhar
2	R\$1500,00	aceita	aceita / aceita	Sem espaço e sem separador
3	R\$_0,99	aceita	aceita / aceita	Parte inteira zero
4	R\$_12.345.678,90	aceita	aceita / aceita	Vários grupos de milhar

5	R\$_1000000,00	aceita	aceita / aceita	Inteiro grande sem separador
6	R\$_7,00	aceita	aceita / aceita	Um único dígito inteiro
7	R\$_0,00	aceita	aceita / aceita	Limite: menor valor caso-limite
8	R\$_999,99	aceita	aceita / aceita	Limite: maior valor sem grupo de milhar caso-limite
9	ε (vazia)	rejeita	rejeita / rejeita	Limite: palavra vazia ε caso-limite
10	R\$_01,50	rejeita	rejeita / rejeita	Zero à esquerda
11	R\$_1.23,45	rejeita	rejeita / rejeita	Grupo de milhar com dois dígitos
12	R\$_1234.567,00	rejeita	rejeita / rejeita	Mistura de formatos (com e sem separador)
13	R\$_10,5	rejeita	rejeita / rejeita	Limite: só um dígito de centavos caso-limite
14	1.234,56	rejeita	rejeita / rejeita	Sem o prefixo R\$
15	R\$_1,234.56	rejeita	rejeita / rejeita	Formato americano
16	R\$_-10,00	rejeita	rejeita / rejeita	Valor negativo

Resultado e limites

Os 16 casos produziram o resultado esperado em re.fullmatch e na simulação do AFNε.

- Não há valor máximo: qualquer quantidade de grupos de milhar é aceita.
- R\$ 0,00 é aceito (frete cortesia); valores negativos e o formato americano (R\$ 1,234.56) são rejeitados.
- Não aceita o símbolo sem o prefixo (1.234,56) nem 'R \$' com espaço entre R e \$.

5.5 ER-05 — AFNε correspondente

$Q = \{q_0, \dots, q_{24}\}$ (25 estados) · $\Sigma = \{ \$, ,, ,, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, R, _ \}$ · estado inicial q_0 (seta "início") · $F = \{q_{20}\}$ (círculo duplo) · 33 transições, 17 movimentos ϵ (tracejados). A tabela δ completa está no Apêndice A.

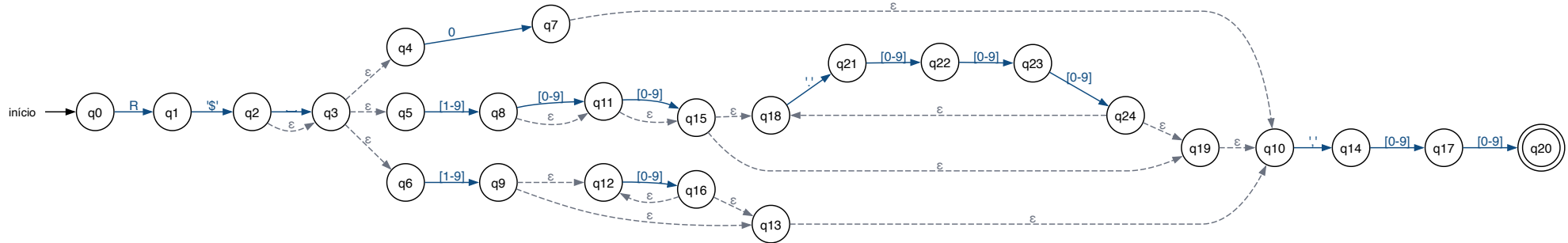


Figura 5 — AFNε de ER-05, gerado pela construção de Thompson a partir de $R \backslash \$? (0 | [1-9] [0-9] \{0, 2\} (\backslash . [0-9] \{3\})^* | [1-9] [0-9]^*) , [0-9] \{2\}$. Versão ampliada: docs/afn/er-05-valor.svg.

6. Testes e análise dos resultados

Os testes foram automatizados com pytest e estão em `tests/`. A execução mais recente resultou em: **223 passed in 0.39s**.

Grupo	O que verifica
Casos das ERs	Os 80 casos das cinco ERs executados com <code>re.fullmatch</code> e, separadamente, simulando o AFNe.
Requisitos mínimos	Cada ER tem ao menos 6 aceitas, 6 rejeitadas e 1 caso-limite.
Alfabeto	O Σ documentado em cada ficha é exatamente o conjunto de símbolos que aparece nas transições do AFNe.
Equivalência AFNe $\times$ regex	3.000 cadeias por ER (15.000 no total), geradas por mutações dos casos de teste (troca, inserção e remoção de símbolos, inclusive fora de Σ), para explorar a fronteira da linguagem. Em todas, AFNe e <code>re.fullmatch</code> concordaram.
Construção de Thompson	Cada operador isolado e combinado; recursos não regulares (<code>\1</code> , <code>lookaround</code> , <code>\d</code> , ponto, âncoras, classe negada) são recusados pelo analisador.
Aplicação	Mensagens de diagnóstico, entradas inválidas, sete arquivos CSV de exemplo (válidos e inválidos) e todas as rotas web.

Resultados por ER

ER	Aceitas	Rejeitadas	Casos-limite	Acertos re	Acertos AFNe
ER-01	8	8	4	16/16	16/16
ER-02	8	8	4	16/16	16/16
ER-03	8	8	3	16/16	16/16
ER-04	8	8	5	16/16	16/16
ER-05	8	8	4	16/16	16/16

Execução com a planilha de exemplo

O arquivo `dados/fretes_exemplo.csv` contém 15 registros. O programa identificou **5 registros totalmente válidos** e 10 com ao menos um campo rejeitado:

ER	Aceitas	Rejeitadas
ER-01 · CNPJ do cliente (alfanumérico ou numérico)	12	3
ER-02 · Placa do veículo (padrão antigo ou Mercosul)	13	2
ER-03 · Telefone do motorista (fixo ou celular, com DDD válido)	12	3
ER-04 · Data da coleta (DD/MM/AAAA com dia compatível com o mês)	12	3
ER-05 · Valor do frete em reais (R\$)	11	4

Exemplos de mensagens produzidas:

Valor	Mensagem
12ABC345/01DE-35	Rejeitada na posição 9: depois de ler "12ABC345", o AFNε não tem transição com o símbolo '/'.
ABC-1D23	Rejeitada na posição 6: depois de ler "ABC-1", o AFNε não tem transição com o símbolo 'D'.
(23)␣91234-5678	Rejeitada na posição 3: depois de ler "(2", o AFNε não tem transição com o símbolo '3'.
31/04/2026	Rejeitada na posição 5: depois de ler "31/0", o AFNε não tem transição com o símbolo '4'.
R\$␣1.23,45	Rejeitada na posição 8: depois de ler "R\$␣1.23", o AFNε não tem transição com o símbolo ','.
33.000.167/0001-01␣	O símbolo ␣ (posição 19) não pertence ao alfabeto Σ desta expressão. Verifique se há espaços sobrando no início ou no fim.

Análise

Todos os casos produziram o resultado esperado, e o AFNε concordou com o motor re em todas as cadeias testadas. Isso confirma, empiricamente, que o padrão do código e o autômato representam a mesma linguagem; a correspondência com a ER formal é garantida pela tabela de equivalência de cada ficha, já que cada atalho da sintaxe (classes, ?, {m}, {m,n}) foi reescrito com os operadores básicos.

Os falsos resultados observados são conhecidos e decorrem de decisões de projeto, e não de erros: a ER descreve o *formato* dos dados. O registro F007 (data 29/02/2027) é aceito pela ER-04, e o CNPJ 00.000.000/0000-00 é aceito pela ER-01 — ambos seriam barrados por uma verificação aritmética (ano bissexto, dígito verificador), que está fora do escopo das expressões.

7. Limitações e melhorias futuras

Limitações

- As ERs validam o formato, não a existência dos dados: dígitos verificadores do CNPJ, placa registrada no Detran e número de telefone ativo não são consultados.
- A ER-04 aceita 29/02 em anos não bissextos. Como o intervalo 1900–2099 é finito, os bissextos poderiam ser descritos por uma ER, mas ela ficaria muito grande.
- A entrada é sensível a maiúsculas/minúsculas e a espaços, de propósito, para que o programa reconheça exatamente a linguagem documentada.
- Os AFNε de Thompson têm muitos estados (84 na ER-03); para leitura humana, um AFD minimizado seria menor.

Melhorias futuras

- Etapa de validação semântica após a ER: dígitos verificadores do CNPJ (módulo 11) e anos bissextos.
- Normalização opcional (maiúsculas, remoção de espaços) com aviso ao usuário, mantendo a ER inalterada.
- Conversão do AFNε em AFD (construção de subconjuntos) e minimização, com visualização lado a lado.
- Leitura direta de planilhas .xlsx e exportação do relatório em .xlsx.
- Novas ERs para outros campos do frete: CEP, chave de acesso da NF-e (44 dígitos) e RNTRC.

8. Contribuições dos integrantes

Integrante	Contribuições
Yuri Aguiar	Arquitetura e organização do repositório; implementação das cinco ERs; construção de Thompson e simulação do AFNε; validação e diagnóstico; processamento de CSV; interface web; testes automatizados; scripts de diagramas, documentação, relatório e slides. Na apresentação: problema, arquitetura, Thompson, ER-03 e demonstração.
Pedro Paulo	Dados de exemplo; levantamento e conferência dos casos de teste das ER-01 e ER-02; revisão das mensagens da interface. Na apresentação: ER-01 e ER-02.
João Rath	Levantamento e conferência dos casos de teste das ER-04 e ER-05; revisão da notação formal e do relatório. Na apresentação: ER-04, ER-05, testes e limitações.

A definição do problema, a escolha das ERs e a revisão dos AFNε foram feitas em conjunto. O registro detalhado está em CONTRIBUIÇÕES.md e no histórico de commits.

9. Uso de Inteligência Artificial

Conforme as orientações do trabalho, declaramos o uso do assistente **Claude Code (Anthropic)** como ferramenta de apoio na sugestão da estrutura do projeto e das páginas web, na

implementação do módulo de construção e simulação do AFNe, na criação dos scripts de geração de diagramas e documentação, na revisão dos casos de teste e na redação inicial do README, deste relatório e dos slides. Todo o conteúdo foi revisado pela equipe, que compreende, explica e é capaz de modificar as expressões, os autômatos e o código apresentados.

10. Conclusão

O ValidaFrete resolve um problema concreto — a validação de cadastros de fretes — usando cinco Expressões Regulares não triviais: a ER-01 acompanha o novo CNPJ alfanumérico; a ER-02 une dois padrões de placa; a ER-03 restringe o DDD aos 67 códigos em uso e só aceita parênteses balanceados, o que exige duplicar a subexpressão do DDD, pois uma ER não tem memória; a ER-04 codifica a quantidade de dias de cada mês; e a ER-05 exige separadores de milhar consistentes.

Ao gerar o AFNe diretamente do padrão do código e comparar autômato e motor de ERs em milhares de cadeias, o trabalho mostra na prática o Teorema de Kleene: a mesma linguagem descrita de forma algébrica (ER), computacional (sintaxe do Python) e operacional (AFNe).

11. Referências

- HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. *Introdução à teoria de autômatos, linguagens e computação*. Rio de Janeiro: Elsevier, 2002.
- MENEZES, P. B. *Linguagens formais e autômatos*. 6. ed. Porto Alegre: Bookman, 2011.
- THOMPSON, K. Regular expression search algorithm. *Communications of the ACM*, v. 11, n. 6, p. 419–422, 1968.
- PYTHON SOFTWARE FOUNDATION. *re — Regular expression operations*. Disponível em: <https://docs.python.org/3/library/re.html>.
- RECEITA FEDERAL DO BRASIL. Instrução Normativa RFB nº 2.229, de 15 de outubro de 2024 (CNPJ alfanumérico).
- ANATEL. Plano Geral de Códigos Nacionais — tabela de DDDs.
- CONTRAN. Resolução nº 729, de 6 de março de 2018 (placas de identificação veicular padrão Mercosul).
- Guia de Sintaxe para Apresentação das Expressões Regulares — material de apoio da disciplina.

Apêndice A – Tabelas de transição dos AFNε

Cada linha representa $\delta(\text{origem}, \text{símbolo}) \ni \text{destino}$. "ε" indica movimento vazio. Classes como [0–9] representam uma transição para cada símbolo da classe.

ER-01 – CNPJ do cliente (alfanumérico ou numérico) (inicial q0, final q32)

Origem	Símbolo	Destino	q8	[A–Z0–9]	q10	q16	[A–Z0–9]	q18	q24	[A–Z0–9]	q26
q0	ε	q1	q9	[A–Z0–9]	q11	q17	[A–Z0–9]	q19	q25	[A–Z0–9]	q27
q1	[A–Z0–9]	q3	q10	[A–Z0–9]	q12	q18	[A–Z0–9]	q20	q26	[0–9]	q28
q2	[A–Z0–9]	q4	q11	[A–Z0–9]	q13	q19	[A–Z0–9]	q21	q27	[A–Z0–9]	q29
q3	[A–Z0–9]	q5	q12	[A–Z0–9]	q14	q20	[A–Z0–9]	q22	q28	[0–9]	q30
q4	[A–Z0–9]	q6	q13	'.'	q15	q21	'/'	q23	q29	[A–Z0–9]	q31
q5	'.'	q7	q14	[A–Z0–9]	q16	q22	[A–Z0–9]	q24	q30	ε	q32
q6	[A–Z0–9]	q8	q15	[A–Z0–9]	q17	q23	[A–Z0–9]	q25	q31	'_'	q33
q7	[A–Z0–9]	q9							q33	[0–9]	q34
									q34	[0–9]	q35
									q35	ε	q32

ER-02 – Placa do veículo (padrão antigo ou Mercosul) (inicial q0, final q15)

Origem	Símbolo	Destino	q3	ε	q5	q7	[A–Z]	q9	q12	[0–9]	q14
q0	[A–Z]	q1	q4	'_'	q6	q8	[0–9]	q10	q13	ε	q15
q1	[A–Z]	q2	q4	ε	q6	q9	[0–9]	q11	q14	ε	q15
q2	[A–Z]	q3	q5	[0–9]	q7	q10	[0–9]	q12			
q3	ε	q4	q6	[0–9]	q8	q11	[0–9]	q13			

ER-03 – Telefone do motorista (fixo ou celular, com DDD válido) (inicial q0, final q83)

Origem	Símbolo	Destino	q5	ε	q12	q7	ε	q22	q16	9	q34
q0	'+'	q1	q5	ε	q13	q7	ε	q23	q17	1	q35
q0	ε	q2	q5	ε	q14	q7	ε	q24	q18	2	q36
q1	5	q3	q5	ε	q15	q7	ε	q25	q19	3	q37
q2	ε	q4	q5	ε	q16	q8	1	q26	q20	4	q38
q2	ε	q5	q6	␣	q2	q9	2	q27	q21	5	q39
q3	5	q6	q6	ε	q2	q10	3	q28	q22	6	q40
q4	'('	q7	q7	ε	q17	q11	4	q29	q23	7	q41
q5	ε	q8	q7	ε	q18	q12	5	q30	q24	8	q42
q5	ε	q9	q7	ε	q19	q13	6	q31	q25	9	q43
q5	ε	q10	q7	ε	q20	q14	7	q32	q26	[1–9]	q44
q5	ε	q11	q7	ε	q21	q15	8	q33	q27	[12478]	q45

q28	[1-578]	q46	q43	[1-9]	q61	q58	ε	q63	q71	[0-9]	q73
q29	[1-9]	q47	q44	ε	q62	q59	ε	q63	q72	[0-9]	q74
q30	[1345]	q48	q45	ε	q62	q60	ε	q63	q73	[0-9]	q75
q31	[1-9]	q49	q46	ε	q62	q61	ε	q63	q74	[0-9]	q76
q32	[134579]	q50	q47	ε	q62	q62	ε	q64	q75	[0-9]	q77
q33	[1-9]	q51	q48	ε	q62	q63	') '	q65	q76	ε	q78
q34	[1-9]	q52	q49	ε	q62	q64	␣	q66	q77	ε	q78
q35	[1-9]	q53	q50	ε	q62	q64	ε	q66	q78	' _ '	q79
q36	[12478]	q54	q51	ε	q62	q65	ε	q64	q78	ε	q79
q37	[1-578]	q55	q52	ε	q62	q66	ε	q67	q79	[0-9]	q80
q38	[1-9]	q56	q53	ε	q63	q66	ε	q68	q80	[0-9]	q81
q39	[1345]	q57	q54	ε	q63	q67	9	q69	q81	[0-9]	q82
q40	[1-9]	q58	q55	ε	q63	q68	[2-5]	q70	q82	[0-9]	q83
q41	[134579]	q59	q56	ε	q63	q69	[0-9]	q71			
q42	[1-9]	q60	q57	ε	q63	q70	[0-9]	q72			

ER-04 — Data da coleta (DD/MM/AAAA com dia compatível com o mês) (inicial q0, final q49)

Origem	Símbolo	Destino	q11	' / '	q15	q22	[13-9]	q28	q36	ε	q39
q0	ε	q1	q12	ε	q16	q23	[0-2]	q29	q37	ε	q39
q0	ε	q2	q13	ε	q16	q24	[13578]	q30	q38	' / '	q40
q0	ε	q3	q14	ε	q17	q25	[02]	q31	q39	ε	q38
q1	ε	q4	q14	ε	q18	q26	0	q32	q40	ε	q41
q1	ε	q5	q15	ε	q19	q27	1	q33	q40	ε	q42
q2	3	q6	q15	ε	q20	q28	ε	q34	q41	1	q43
q3	3	q7	q16	' / '	q21	q29	ε	q34	q42	2	q44
q4	0	q8	q17	0	q22	q30	ε	q35	q43	9	q45
q5	[12]	q9	q18	1	q23	q31	ε	q35	q44	0	q46
q6	0	q10	q19	0	q24	q32	[1-9]	q36	q45	ε	q47
q7	1	q11	q20	1	q25	q33	[0-2]	q37	q46	ε	q47
q8	[1-9]	q12	q21	ε	q26	q34	ε	q38	q47	[0-9]	q48
q9	[0-9]	q13	q21	ε	q27	q35	ε	q38	q48	[0-9]	q49
q10	' / '	q14									

ER-05 — Valor do frete em reais (R\$) (inicial q0, final q20)

Origem	Símbolo	Destino	q5	[1-9]	q8	q11	ε	q15	q18	' . '	q21
q0	R	q1	q6	[1-9]	q9	q12	[0-9]	q16	q19	ε	q10
q1	' \$ '	q2	q7	ε	q10	q13	ε	q10	q21	[0-9]	q22
q2	␣	q3	q8	[0-9]	q11	q14	[0-9]	q17	q22	[0-9]	q23
q2	ε	q3	q8	ε	q11	q15	ε	q18	q23	[0-9]	q24
q3	ε	q4	q9	ε	q12	q15	ε	q19	q24	ε	q18
q3	ε	q5	q9	ε	q13	q16	ε	q12	q24	ε	q19
q3	ε	q6	q10	' , '	q14	q16	ε	q13			
q4	0	q7	q11	[0-9]	q15	q17	[0-9]	q20			